DESIGN PROJECT – III · SEMESTER V

ThreatScope

A Reproducible SOC Detection Lab with Ground-Truth Rule Evaluation

A project report submitted in partial fulfilment of the requirements of
Design Project – III (Semester V) for the degree of

BACHELOR OF TECHNOLOGY

with specialisation in Cyber Security

Submitted by

Malepati Sree Harsha

Register No. 23CU0340018

DISC

HINDUSTAN INSTITUTE OF TECHNOLOGY AND SCIENCE

(Deemed to be University)

Padur, Kelambakkam, Chennai – 603 103

Academic Year 2025–26

DECLARATION

I, **Malepati Sree Harsha** (Register No. 23CU0340018), hereby declare that the project report entitled **“ThreatScope: A Reproducible SOC Detection Lab with Ground-Truth Rule Evaluation”**, submitted as Design Project – III in Semester V of the Bachelor of Technology programme with specialisation in Cyber Security, DISC, Hindustan Institute of Technology and Science, during the academic year 2025–26, is a record of my own work.

Where the work of others has been used, it is acknowledged and cited in the references. All experiments were performed on systems I own, using bundled or synthetic data, and no third-party system was tested without authorization.

Place: Chennai

Date: _____

Malepati Sree Harsha

Register No. 23CU0340018

ACKNOWLEDGEMENT

I thank the faculty of DISC, Hindustan Institute of Technology and Science, for the course structure, laboratory facilities and feedback that shaped this design project, and the institution for the opportunity to work on an applied security problem.

I also acknowledge the open-source communities behind Python, Flask, scikit-learn, the cryptography library, OpenSSL, John the Ripper, Wireshark/TShark, Suricata, Nmap and OWASP ZAP, whose tools and documentation made this work possible. Finally, I thank my family and friends for their support.

Malepati Sree Harsha

ABSTRACT

Security operations centres (SOCs) correlate authentication, firewall, web, endpoint and network evidence into alerts that analysts triage. Commercial and open-source SIEM platforms are powerful but heavy, and detection rules are often deployed without a reproducible measurement of how many incidents they catch or how much noise they produce. This project, ThreatScope, is a compact SOC lab that parses four log types and packet captures, applies five detection rules mapped to MITRE ATT&CK (brute force, anomalous valid-account login, network service scanning, large data egress and suspicious command execution), and presents alerts with evidence and response playbooks in an interactive console.

The lab integrates industry tools safely: TShark and Suricata analyse a bundled PCAP offline, while Nmap and OWASP ZAP run only against a demo container on an internal Docker network with no route out. The research contribution is a **ground-truth benchmark**: because the log generator injects known incidents into random benign noise, every alert can be scored as a true or false positive. Over 25 seeds the rules found all 150 injected incidents with 0 false positives, and threshold sweeps locate each rule's safe operating band (for example, brute-force thresholds of 3–16 failures per 120 s are exact on this data; 2 adds noise and 20 misses the attack). The system is hardened with a strict Content-Security-Policy, OWASP headers, audit logging, rate limits and optional bearer-token authentication, and is verified by 13 automated tests and load testing.

Keywords: security operations, SIEM, detection engineering, MITRE ATT&CK, intrusion detection, Suricata, TShark, Nmap, OWASP ZAP, ground-truth evaluation

CONTENTS

Declaration ii
Acknowledgement iii
Abstract iv
List of Figures vii
List of Tables viii
Chapter 1 Introduction 1
1.1 Background 1
1.2 Motivation 1
1.3 Problem Statement 1
1.4 Objectives 1
1.5 Scope and Ethics 2
1.6 Organisation of the Report 2
Chapter 2 Literature Survey 3
2.1 Intrusion Detection Foundations 3
2.2 SIEM, Log Management and Incident Response 3
2.3 Adversary Frameworks 3
2.4 Network and Web Assessment Tools 3
2.5 Comparison and Research Gap 3
Chapter 3 System Requirements Specification 5
3.1 Functional Requirements 5
3.2 Non-Functional Requirements 5
3.3 Hardware and Software Requirements 6
Chapter 4 System Design 7
4.1 Architecture 7
4.2 Data Flow Diagrams 8
4.3 UML Diagrams 9
4.4 Deployment and Network Isolation 10
4.5 Detection Rules 11
Chapter 5 Methodology and Implementation 12
5.1 Technology Stack 12
5.2 Stage 1 — Synthetic Telemetry with Known Incidents 12
5.3 Stage 2 — Parsers 12
5.4 Stage 3 — Sliding-Window Detection Rules 12
5.5 Stage 4 — Packet Analysis 13
5.6 Stage 5 — Isolated Active Tools 13
5.7 Stage 6 — Ground-Truth Benchmark (Novel Component) 13

5.8 Stage 7 — Analyst Console 14
5.9 Stage 8 — Platform Security 14
Chapter 6 Testing and Validation 16
6.1 Strategy 16
6.2 Automated Test Cases 16
6.3 Functional and Non-Functional Checks 17
Chapter 7 Results and Discussion 18
7.1 Analyst Console 18
7.2 Real Tool Integration 20
7.3 Ground-Truth Detection Quality 21
7.4 Threshold Sensitivity 22
7.5 Performance 23
7.6 Mobile Layout 24
7.7 Limitations 24
Chapter 8 Conclusion and Future Scope 26
8.1 Conclusion 26
8.2 Future Scope 26
References 27
Appendix A Installation and User Manual 28
Appendix B Demonstration Script 29
Appendix C API Reference 30
Appendix D Repository Structure 31

LIST OF FIGURES

Figure 4.1 Layered architecture of ThreatScope 7
Figure 4.2 DFD level 0 (context diagram) 8
Figure 4.3 DFD level 1 8
Figure 4.4 Use-case diagram 9
Figure 4.5 Sequence diagram for a dashboard replay 9
Figure 4.6 Class and module diagram 10
Figure 4.7 Activity diagram of the sliding-window brute-force rule 10
Figure 4.8 Deployment diagram with isolated lab network 11
Figure 7.1 SOC overview: 918 events, 6 alerts across 5 ATT&CK techniques 18
Figure 7.2 Alert triage with evidence and response playbook 19
Figure 7.3 Bundled synthetic evidence by source 19
Figure 7.4 TShark (62 packets) and Suricata (3 alerts) on the bundled PCAP 20
Figure 7.5 Nmap connect scan of the isolated demo target 20
Figure 7.6 OWASP ZAP spider and passive baseline 21
Figure 7.7 Detection-quality benchmark in the console 22
Figure 7.8 Precision and recall versus threshold 23
Figure 7.9 Latency per endpoint 24
Figure 7.10 Console at 390 px width 24

LIST OF TABLES

Table 2.1 Existing platforms compared with ThreatScope....3
Table 3.1 Functional requirements....5
Table 3.2 Non-functional requirements....5
Table 3.3 Hardware and software requirements....6
Table 4.1 Rules, ATT&CK mapping and logic....11
Table 5.1 Technology choices....12
Listing 5.2 Brute-force rule (threatscope/detect.py)....12
Listing 5.3 Benchmark scoring (threatscope/benchmark.py)....13
Listing 5.4 Optional token authentication (threatscope/app.py)....14
Table 6.1 Test levels....16
Table 6.2 Automated test cases and results....16
Table 6.3 Functional, safety, usability and performance checks....17
Table 7.1 ZAP passive findings on the demo target (16 alerts)....21
Table 7.2 Per-rule results over 25 seeds (default thresholds)....21
Table 7.3 Brute-force threshold sweep (failures per 120 s)....22
Table 7.4 Port-scan threshold sweep (distinct ports per 60 s)....22
Table 7.5 Load-test results (60 requests, 4 workers)....23

CHAPTER 1

INTRODUCTION

1.1 Background

A security operations centre collects logs from many sources, applies detection logic, and gives analysts alerts to investigate and respond to [3], [4]. Detection logic ranges from signatures on network traffic (Snort [6], Suricata [9]) and event correlation in SIEM platforms to anomaly models dating back to Denning's intrusion-detection model [5]. MITRE ATT&CK [1], [2] provides a shared vocabulary of adversary tactics and techniques, against which detections are now routinely mapped.

1.2 Motivation

For students, a production SIEM is difficult to study: it needs real telemetry, licences or substantial hardware, and its rules are hard to inspect. More fundamentally, detection rules are often tuned by intuition. Sommer and Paxson [7] argue that intrusion-detection evaluation suffers from a lack of ground truth and from evaluations that do not reflect operational conditions. A learning lab should make rules readable and should *measure* them.

Active tools such as Nmap and ZAP introduce a second concern: scanning systems without authorization is illegal and harmful. A teaching lab must make that impossible by construction, not just by policy.

1.3 Problem Statement

To build a reproducible, safe SOC lab that ingests multi-source logs and packet captures, raises ATT&CK-mapped alerts with evidence and playbooks, integrates real network-security tools without any possibility of scanning third parties, and evaluates every detection rule against ground truth, including its sensitivity to threshold choices.

1.4 Objectives

1. Parse authentication, firewall, web and process logs into typed events; tolerate malformed lines.
2. Implement five readable detection rules with sliding windows, mapped to MITRE ATT&CK, each with a response playbook.
3. Analyse packet captures with TShark and Suricata offline.
4. Run Nmap and OWASP ZAP only against an isolated demo target on an internal network.

5. Build a ground-truth benchmark over many seeds and threshold sweeps for tunable rules.
6. Provide an analyst console with KPIs, alert queue, evidence, playbooks, uploads and report export.
7. Harden the platform (CSP, headers, audit logging, rate limits, optional token authentication) and test it.

1.5 Scope and Ethics

All bundled telemetry is synthetic and uses documentation address ranges (RFC 5737 [15]). Scan targets are constants in the backend; the scanner, ZAP and demo target share a Docker network declared `internal: true`, so even a modified request cannot reach the internet or the host LAN. Uploaded files are processed in memory or in temporary files that are deleted after analysis.

1.6 Organisation of the Report

Chapter 2 surveys SIEM and intrusion-detection work. Chapter 3 gives requirements, Chapter 4 the design, Chapter 5 the implementation, Chapter 6 testing, Chapter 7 results and Chapter 8 conclusions.

CHAPTER 2

LITERATURE SURVEY

2.1 Intrusion Detection Foundations

Denning [5] proposed modelling normal behaviour and flagging deviations. Signature systems such as Snort [6] and Bro/Zeek [8] match known patterns in traffic. Khraisat et al. [17] survey signature- and anomaly-based approaches and the datasets used to evaluate them. Sommer and Paxson [7] caution that anomaly detection in operational networks is hard to evaluate and that false positives are costly for analysts.

2.2 SIEM, Log Management and Incident Response

NIST SP 800-92 [4] describes log management infrastructure; NIST SP 800-61 [3] defines the incident-handling lifecycle (preparation, detection and analysis, containment, eradication and recovery, post-incident activity). Open-source platforms such as Wazuh [14] and vendor SIEMs implement these at scale. Sigma [13] provides a vendor-neutral rule format, encouraging detection-as-code.

2.3 Adversary Frameworks

MITRE ATT&CK [1], [2] catalogues tactics (the adversary's goal) and techniques (how it is achieved), e.g. T1110 Brute Force and T1046 Network Service Discovery. Mapping alerts to techniques helps coverage analysis and communication, though a mapping is not evidence that a detection is effective.

2.4 Network and Web Assessment Tools

TShark [10] is the command-line form of Wireshark. Suricata [9] is a multi-threaded IDS/IPS engine with a rule language and EVE JSON output. Nmap [11] performs host and service discovery. OWASP ZAP [12] is a web application scanner whose spider and passive scan inspect responses without attacking.

2.5 Comparison and Research Gap

Table 2.1: Existing platforms compared with ThreatScope

Platform	Strength	Limitation for learning / evaluation
Commercial SIEM	Scale, integrations	Cost, opacity; no ground truth for rules
Wazuh / Security Onion [14]	Open source, complete	Heavy deployment; real data needed

Platform	Strength	Limitation for learning / evaluation
Snort / Suricata / Zeek [6], [8], [9]	Mature network detection	Network only; no cross-source correlation
Sigma [13]	Portable rule format	No bundled benchmark data
Public IDS datasets [17]	Labelled traffic	Often criticised as unrepresentative [7]
ThreatScope (this work)	Small, readable, safe	**Ground-truth benchmark + threshold sweeps per rule**

The gap is a learner-scale lab in which every rule ships with a reproducible, ground-truth measurement and a threshold-sensitivity analysis, alongside real tools that are isolated by design. The contribution is methodological and educational; the benchmark validates rules against the scenarios they were written for and does not estimate real-world detection rates.

CHAPTER 3

SYSTEM REQUIREMENTS SPECIFICATION

3.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement	Endpoint / module
FR-1	Parse auth, firewall, web and process log lines into typed events	detect.parse_*
FR-2	Detect brute force (T1110) with a sliding window and tunable threshold	detect_brute_force
FR-3	Detect anomalous internal→external login for one user (T1078)	detect_impossible_travel
FR-4	Detect port scans by distinct ports per window (T1046)	detect_port_scan
FR-5	Detect large data egress per source (T1041)	detect_exfil
FR-6	Detect suspicious commands and web-shell parents (T1059)	detect_suspicious_process
FR-7	Summarise alerts: severity counts, timeline, top entities, techniques	GET /api/dashboard
FR-8	Analyse PCAPs with TShark and Suricata	nettools.tshark_summary / suricata_alerts
FR-9	Ingest uploaded logs and PCAPs (≤ 8 files, ≤ 10 MB each)	POST /api/upload
FR-10	Run Nmap and ZAP against the fixed demo target only	POST /api/nmap-demo, /api/zap-demo
FR-11	Score rules against ground truth with threshold sweeps	GET /api/benchmark
FR-12	Export / print the current report	Browser UI

3.2 Non-Functional Requirements

Table 3.2: Non-functional requirements

ID	Category	Requirement	Verification
NFR-1	Safety	No scan can target anything but demo-target	Constant targets; internal network; test
NFR-2	Security	CSP/OWASP headers; rate limits on scans and uploads	tests/test_security.py
NFR-3	Security	Optional bearer-token authentication	test_optional_api_token
NFR-4	Privacy	Uploaded evidence is never persisted	Code review; temp file deletion
NFR-5	Accountability	Audit log of scans and auth failures	Log inspection
NFR-6	Performance	Dashboard incl. TShark + Suricata p95 < 500 ms	scripts/loadtest.py
NFR-7	Reproducibility	Deterministic fixtures and seeded benchmark	make evaluate / benchmark
NFR-8	Usability	Responsive from 320 px; 44 px touch targets	Mobile screenshot

3.3 Hardware and Software Requirements

Table 3.3: Hardware and software requirements

Item	Minimum	Used
CPU / RAM	64-bit, 8 GB RAM (ZAP needs ~1.5 GB)	Apple Silicon
Disk	≈ 3 GB for images	—
Runtime	Docker Desktop with Compose	Docker 29
Tools in image	tshark, suricata, nmap, openssl	Debian packages
ZAP	ghcr.io/zaproxy/zaproxy:stable	Separate container

CHAPTER 4

SYSTEM DESIGN

4.1 Architecture

Parsing, detection, benchmarking and tool adapters are separate modules behind a Flask API (Figure 4.1). A false match can therefore be traced from the alert to the rule and the input lines that triggered it.

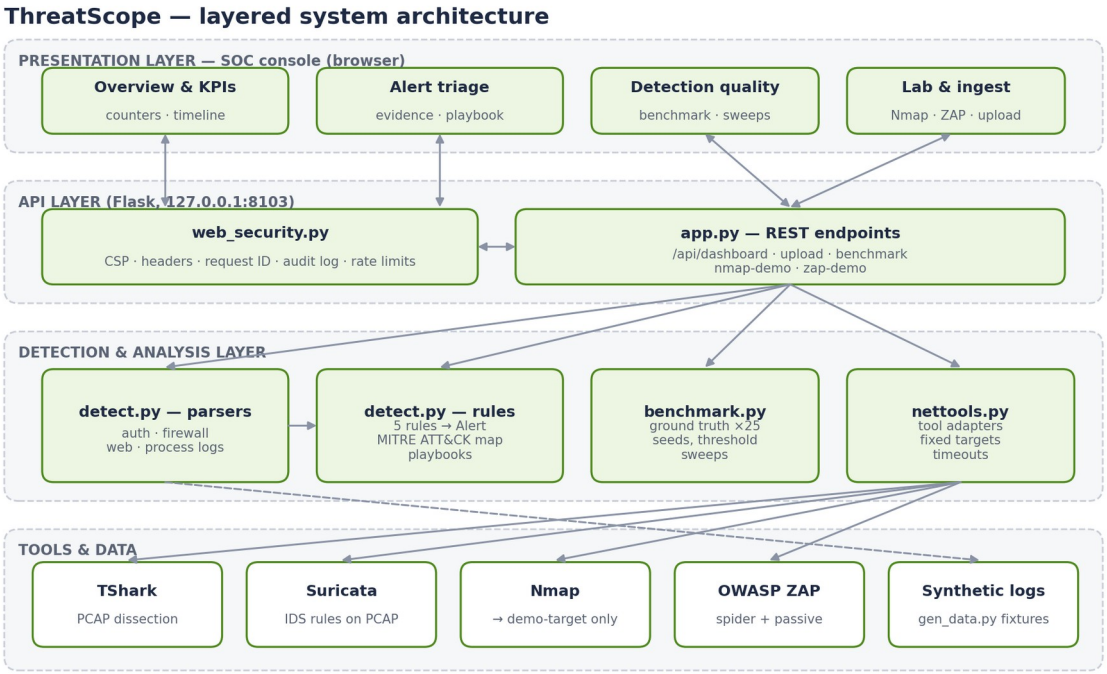


Figure 4.1: Layered architecture of ThreatScope

4.2 Data Flow Diagrams

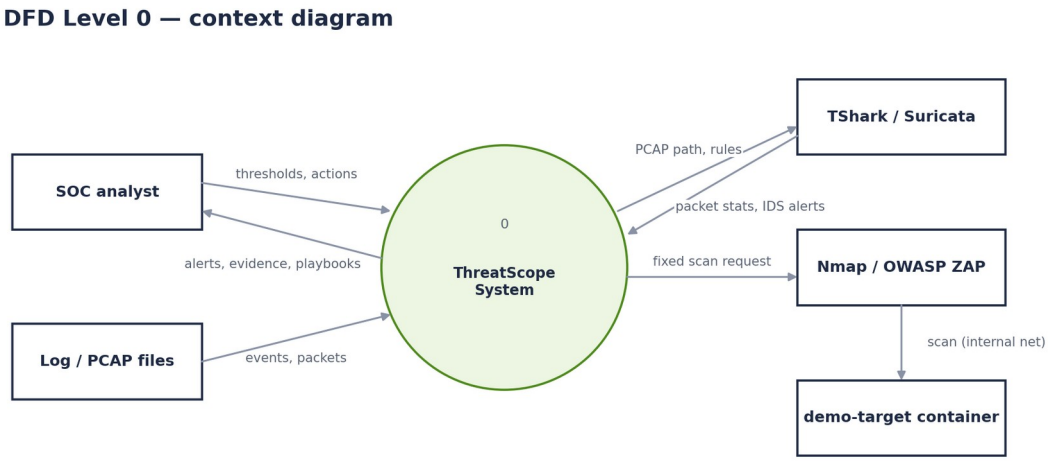


Figure 4.2: DFD level 0 (context diagram)

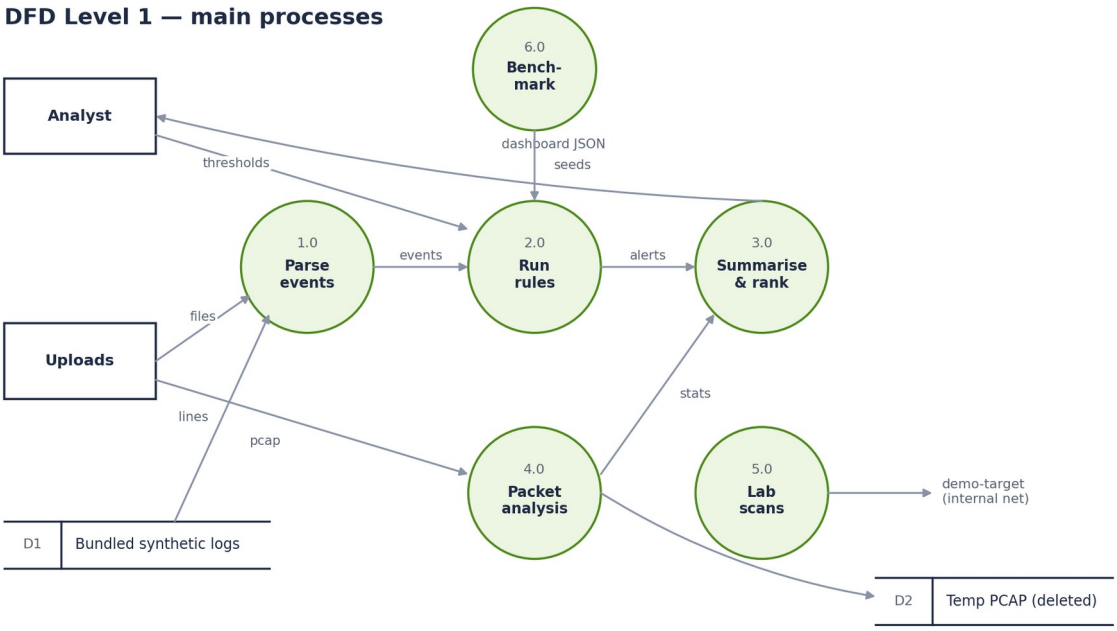


Figure 4.3: DFD level 1

4.3 UML Diagrams

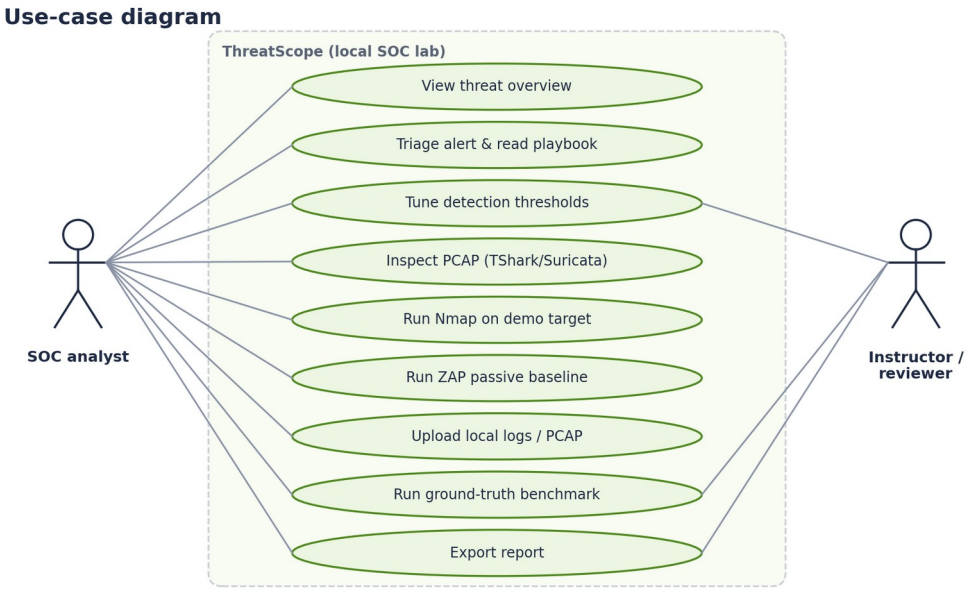


Figure 4.4: Use-case diagram

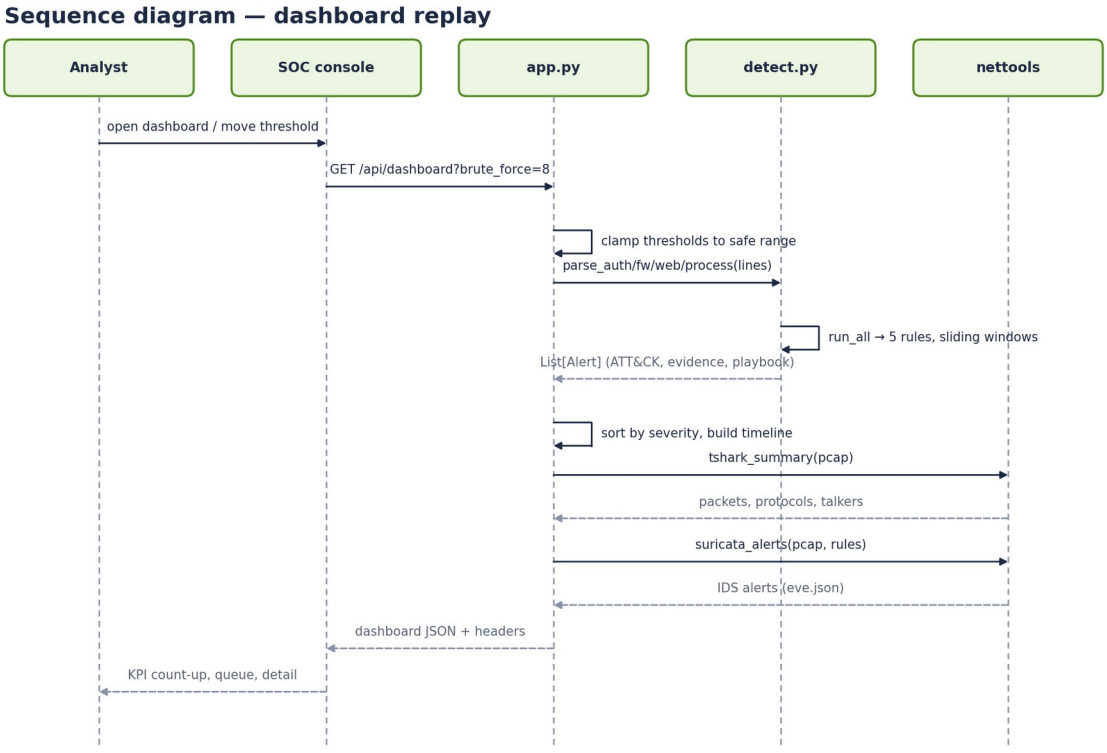


Figure 4.5: Sequence diagram for a dashboard replay

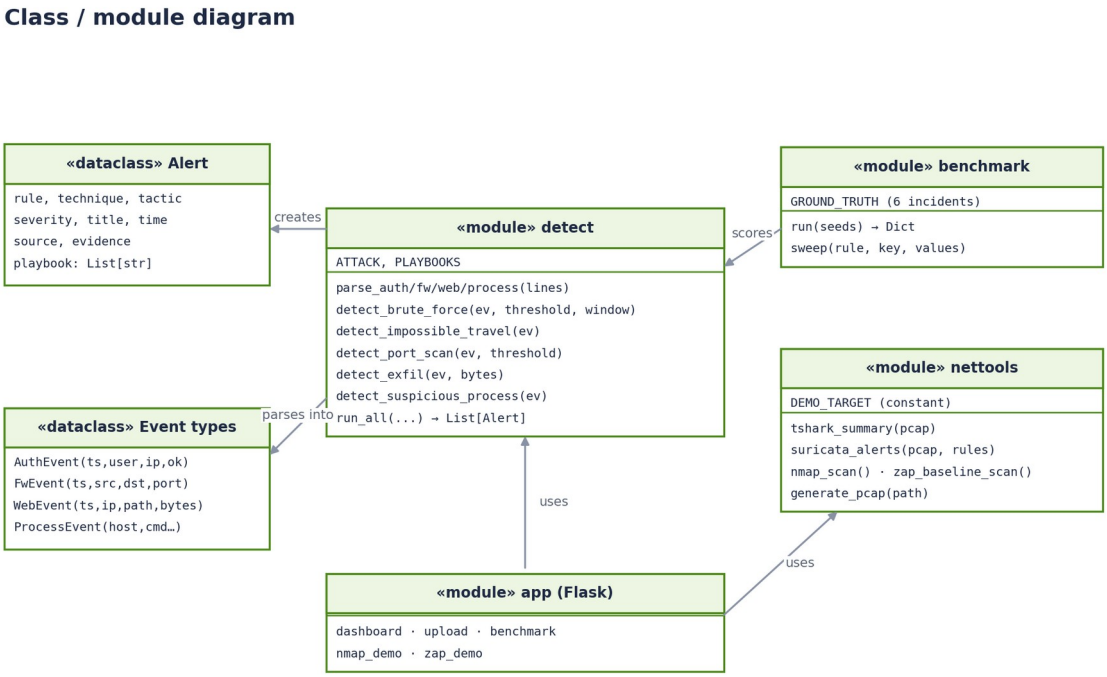


Figure 4.6: Class and module diagram

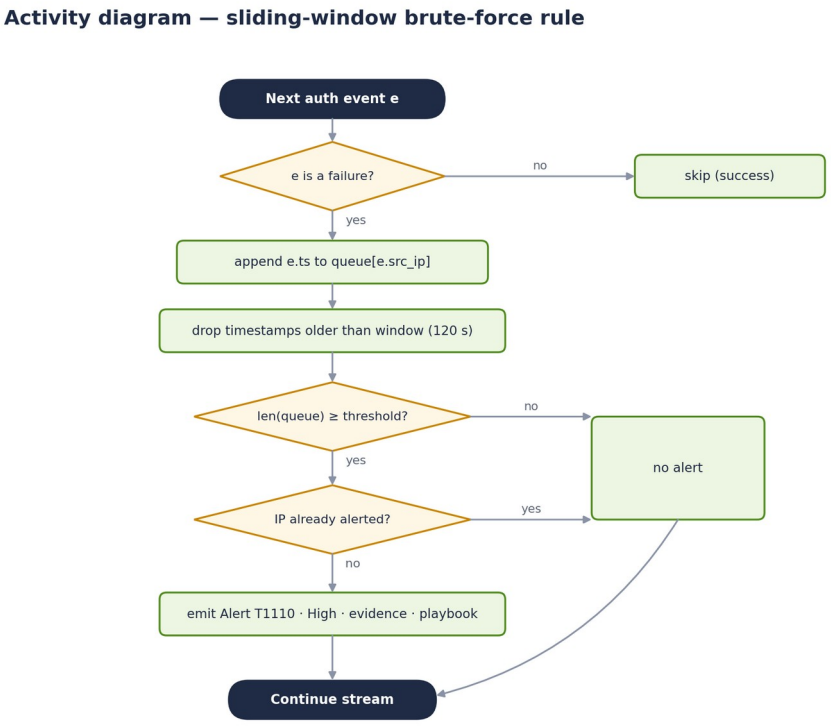


Figure 4.7: Activity diagram of the sliding-window brute-force rule

4.4 Deployment and Network Isolation

Three containers run under Compose (Figure 4.9). Only the dashboard joins the ordinary `web` bridge network, so that its loopback port mapping works; the demo target and ZAP exist only

on `lab`, which is declared `internal: true` and has no route out. All containers use `no-new-privileges` and memory limits.

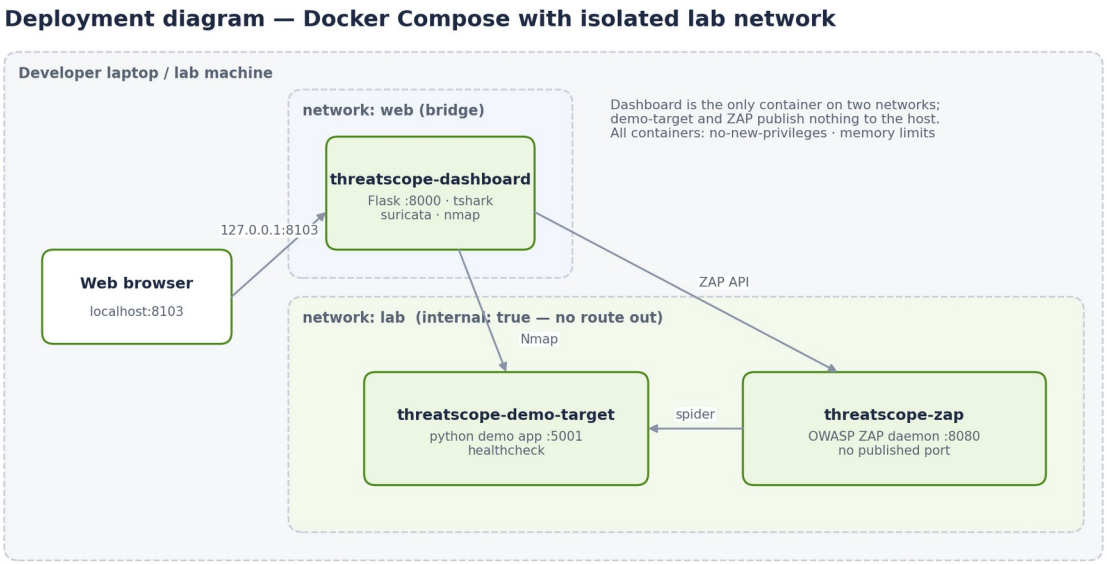


Figure 4.8: Deployment diagram with isolated lab network

4.5 Detection Rules

Table 4.1: Rules, ATT&CK mapping and logic

Rule	ATT&CK	Severity	Logic
brute_force_login	T1110 Brute Force	High	≥ threshold (default 8) failures from one IP within 120 s
impossible_travel	T1078 Valid Accounts	High	Same user, successful logins from internal and external IPs within 600 s
port_scan	T1046 Network Service Discovery	Medium	≥ threshold (default 15) distinct destination ports from one IP within 60 s
data_exfil	T1041 Exfiltration over C2	Critical	≥ 5 MB transferred by one source in the web log
suspicious_process	T1059 Command and Scripting Interpreter	High	Encoded PowerShell, curl/wget, reverse shells, or a shell spawned by a web server

CHAPTER 5

METHODOLOGY AND IMPLEMENTATION

5.1 Technology Stack

Table 5.1: Technology choices

Technology	Role	Justification
Python 3.12, Flask	API and detection engine	Readable rules; easy to test
TShark, Suricata	Packet analysis and IDS	Industry-standard, offline on PCAP
Nmap, OWASP ZAP	Active assessment (isolated)	Real tools; scope fixed by design
scapy	Deterministic PCAP generation	Reproducible teaching capture
Docker Compose	Isolation and packaging	Internal network enforces scope
Vanilla JS console	Analyst UI	No build chain; strict CSP
pytest, GitHub Actions	Quality	Rules tested on every push

5.2 Stage 1 — Synthetic Telemetry with Known Incidents

`gen_data.py` writes deterministic logs: 200 benign logins (15% failures), 300 firewall events, 250 web requests and 90 process events among internal hosts, plus injected incidents: 16 failed logins from 203.0.113.9, a login for *bob* from an internal then an external address, a 28-port scan from 198.51.100.7, 30 large exports by 10.0.0.23 and two suspicious commands. The bundled set holds 918 events (218 auth, 328 firewall, 280 web, 92 process).

5.3 Stage 2 — Parsers

Each parser applies one anchored regular expression per line and builds a typed dataclass; lines that do not match, or carry invalid timestamps, are skipped rather than crashing the pipeline. This tolerance is tested explicitly.

5.4 Stage 3 — Sliding-Window Detection Rules

Windowed rules keep a per-source deque of timestamps and drop entries older than the window on each event, so each event is processed in amortised O(1) time. An alert fires once per source to avoid alert storms. Every alert carries its ATT&CK technique and tactic, the evidence string and a playbook aligned with the NIST incident-handling lifecycle [3].

Listing 5.2: Brute-force rule (threatscope/detect.py)

```
def detect_brute_force(events: List[AuthEvent], threshold=8, window=120) -> List[Alert]:
    fails: Dict[str, deque] = defaultdict(deque)
    fired, alerts = set(), []
    for e in events:
```

```
        if e.ok:
            continue
        q = fails[e.src_ip]; q.append(e.ts)
        while q and (e.ts - q[0]) > timedelta(seconds=window):
            q.popleft()
        if len(q) >= threshold and e.src_ip not in fired:
            fired.add(e.src_ip)
            alerts.append(_mk("brute_force_login", "T1110", "High", "Password brute force",
                              e.ts, e.src_ip, f"{len(q)} failed logins from {e.src_ip} within
{window}s"))
    return alerts
```

5.5 Stage 4 — Packet Analysis

A deterministic PCAP is generated with scapy (a 30-port SYN sweep and HTTP requests including /export/ paths to an external address). TShark summarises protocols, ports and talkers; Suricata runs two lab signatures with EVE JSON output. On the bundled capture TShark parsed 62 packets and Suricata raised 3 alerts.

5.6 Stage 5 — Isolated Active Tools

Nmap and ZAP adapters take no target argument: the target is the constant `demo-target`. The Compose file places the dashboard, ZAP and the demo target on an internal network, so the scope is enforced by the network as well as by the code. Nmap uses a TCP connect scan; ZAP runs a spider and a passive scan only, never an active attack.

5.7 Stage 6 — Ground-Truth Benchmark (Novel Component)

What: `benchmark.py` regenerates the logs with 25 different seeds (varying all benign noise), runs every rule, and labels each alert as a true positive if its (rule, source) pair is one of the six injected incidents, otherwise a false positive; missed incidents are false negatives. It then sweeps the brute-force and port-scan thresholds. **Why:** this turns rule tuning from intuition into measurement, in the spirit of detection-as-code. **How:** precision = TP/(TP+FP), recall = TP/(TP+FN), aggregated over seeds.

Listing 5.3: Benchmark scoring (threatscope/benchmark.py)

```
def _score(alerts) -> Dict[str, Dict[str, int]]:
    found = {(a.rule, a.source) for a in alerts}
    per_rule = {}
    for rule in RULES:
        truth = {s for r, s in GROUND_TRUTH if r == rule}
        got = {s for r, s in found if r == rule}
        per_rule[rule] = {"tp": len(truth & got), "fp": len(got - truth), "fn": len(truth -
got)}
    return per_rule

def run(seeds: int = 25) -> Dict:
    totals = defaultdict(lambda: {"tp": 0, "fp": 0, "fn": 0})
    for seed in range(1, seeds + 1):
        auth, fw, web, proc = _events(seed)
        for rule, c in _score(run_all(auth, fw, web, proc=proc)).items():
            for k in c:
                totals[rule][k] += c[k]
    per_rule = []
```

```
    for rule in RULES:
        t = totals[rule]
        p, r = _rates(t["tp"], t["fp"], t["fn"])
        per_rule.append({"rule": rule, **t, "precision": p, "recall": r})
    tp = sum(x["tp"] for x in per_rule); fp = sum(x["fp"] for x in per_rule)
    fn = sum(x["fn"] for x in per_rule)
    p, r = _rates(tp, fp, fn)

    def sweep(rule: str, key: str, values: List[int]):
        rows = []
        for v in values:
            agg = {"tp": 0, "fp": 0, "fn": 0}
            for seed in range(1, seeds + 1):
                auth, fw, web, proc = _events(seed)
                c = _score(run_all(auth, fw, web, proc=proc, **{key: v}))[rule]
                for k in agg:
                    agg[k] += c[k]
            pp, rr = _rates(**agg)
    # … (trimmed; see repository for the full function)
```

5.8 Stage 7 — Analyst Console

The console shows animated KPI counters, an hourly alert timeline, a severity-ordered queue, an evidence and playbook panel, top entities, ATT&CK coverage, replay controls for thresholds, packet-inspection results, the isolated scan lab, uploads and the detection-quality benchmark with threshold charts. The layout is responsive; below 720 px the sidebar becomes a scrollable tab strip and all controls are at least 44 px tall.

5.9 Stage 8 — Platform Security

The `web_security` layer provides CSP and OWASP headers, request IDs, body-free access logs, JSON errors and rate limits (Nmap 4/min, ZAP 2/min, uploads 20/min, benchmark 6/min); scans are written to an audit log. Optional bearer-token authentication protects every `/api/` route when `THREATSCOPE_API_TOKEN` is set, using a constant-time comparison and logging failures without the token value. A Compose networking defect was also fixed: with the dashboard only on the internal network, Docker silently dropped its port mapping, so the console was unreachable; it now joins one extra non-internal network while the scan targets remain isolated.

Listing 5.4: Optional token authentication (threatscope/app.py)

```
@app.before_request
def _require_token():
    """Optional bearer-token authentication for every /api/ route.

    Off by default (the dashboard binds to loopback for a single analyst). Set
    THREATSCOPE_API_TOKEN to require ``Authorization: Bearer <token>``, e.g. when
    the dashboard is shared on a lab network. The comparison is constant-time
    and failed attempts are written to the audit log without the token value.
    """
    expected = os.getenv("THREATSCOPE_API_TOKEN", "")
    if not expected or not request.path.startswith("/api/"):
        return None
    supplied = request.headers.get("Authorization", "")
    if hmac.compare_digest(supplied.encode(), f"Bearer {expected}".encode()):
        return None
    log.warning("rid=%s auth failure path=%s client=%s", getattr(g, "rid", "-"),
request.path, request.remote_addr)
```

```
    response = jsonify({"error": "authentication required"})
    response.status_code = 401
    response.headers["WWW-Authenticate"] = 'Bearer realm="threatscope"'
    return response
```

CHAPTER 6

TESTING AND VALIDATION

6.1 Strategy

Table 6.1: Test levels

Level	Scope	Tool
Unit	Parsers and rules	pytest
Integration	API routes incl. uploads	Flask test client
Security	Scan scope, headers, auth, rate limits, log injection	pytest
Detection validation	Ground-truth benchmark, 25 seeds; threshold sweeps	benchmark.py
Functional	Real tools in Docker; browser flows	Playwright + screenshots
Performance	Throughput and latency	scripts/loadtest.py

6.2 Automated Test Cases

All 13 automated tests pass.

Table 6.2: Automated test cases and results

ID	Type	Scenario	Expected	Actual (test function)
TC-01	Integration	Bundled fixtures parse and detect	All five rules fire on the fixture	**PASS** (test_bundled_log_fixtures_parse_and_detect)
TC-02	Unit	Malformed rows	Skipped without exceptions	**PASS** (test_parsers_skip_malformed_rows)
TC-03	Unit	Threshold changes	Higher thresholds suppress matches	**PASS** (test_detection_thresholds_change_expected_matches)
TC-04	Security	Nmap target cannot be changed	Arbitrary targets rejected	**PASS** (test_nmap_rejects_arbitrary_targets)
TC-05	Integration	Dashboard and local-only scan API	Summary, techniques; scans scoped	**PASS** (test_dashboard_and_local_only_scan_api)
TC-06	Security	Uploads in memory; bad files rejected	Parsed counts; 400 for unsupported	**PASS** (test_upload_logs_is_in_memory_and_bad_files_are_rejected)
TC-07	Validation	Ground-truth benchmark	All injected incidents, 0 FP; threshold 20 misses	**PASS** (test_benchmark_finds_every_injected_incident_without_noise_alerts)
TC-08	Security	Security headers	CSP, DENY, nosniff, no-referrer	**PASS** (test_security_headers_on_page_and_api)

ID	Type	Scenario	Expected	Actual (test function)
TC-09	Security	API not cached	Cache-Control: no-store	**PASS** (test_api_responses_are_not_cached)
TC-10	Security	Request-ID log-injection guard	Forged ID replaced	**PASS** (test_request_id_is_echoed_only_when_safe)
TC-11	Security	Unknown API route	JSON 404	**PASS** (test_unknown_api_route_returns_json_error)
TC-12	Security	Rate limit on Nmap	5th call within a minute → 429	**PASS** (test_expensive_endpoint_is_rate_limited)
TC-13	Security	Optional token authentication	401 without/with wrong token; 200 with token; headers kept	**PASS** (test_optional_api_token)

6.3 Functional and Non-Functional Checks

Table 6.3: Functional, safety, usability and performance checks

ID	Type	Scenario	Expected	Actual
TC-14	Functional	TShark on bundled PCAP (Docker)	62 packets; TCP/HTTP split	**PASS** (Fig. 7.3)
TC-15	Functional	Suricata on bundled PCAP	3 alerts incl. export-path signature	**PASS** (Fig. 7.3)
TC-16	Functional	Nmap demo scan	1 open port on demo-target (5001/tcp)	**PASS** (Fig. 7.4)
TC-17	Functional	ZAP spider + passive scan	16 passive alerts, spider 100%	**PASS** (Fig. 7.5)
TC-18	Safety	Lab network isolation	demo-target/ZAP publish no host ports; lab internal	**PASS** (compose config)
TC-19	Usability	Console at 390 px	No horizontal overflow (was broken before fix)	**PASS** (Fig. 7.8)
TC-20	Performance	Load test	0 errors; dashboard p95 < 500 ms	**PASS** (Table 7.4)

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Analyst Console

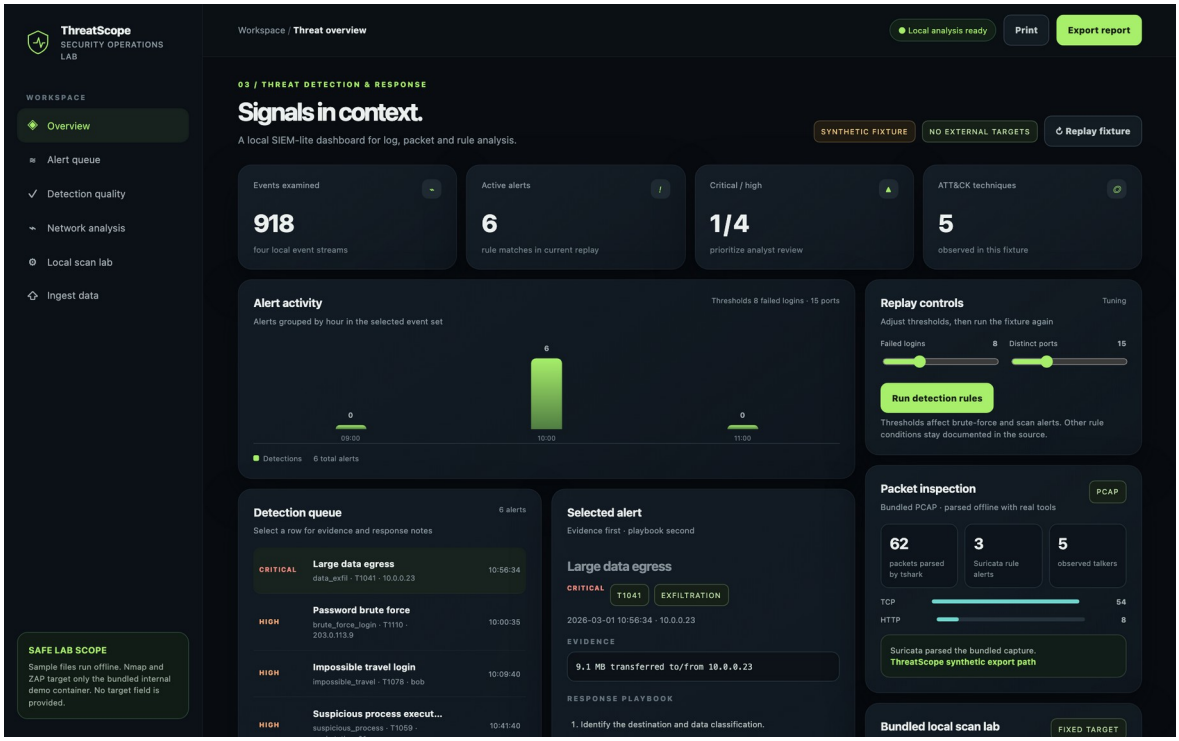


Figure 7.1: SOC overview: 918 events, 6 alerts across 5 ATT&CK techniques

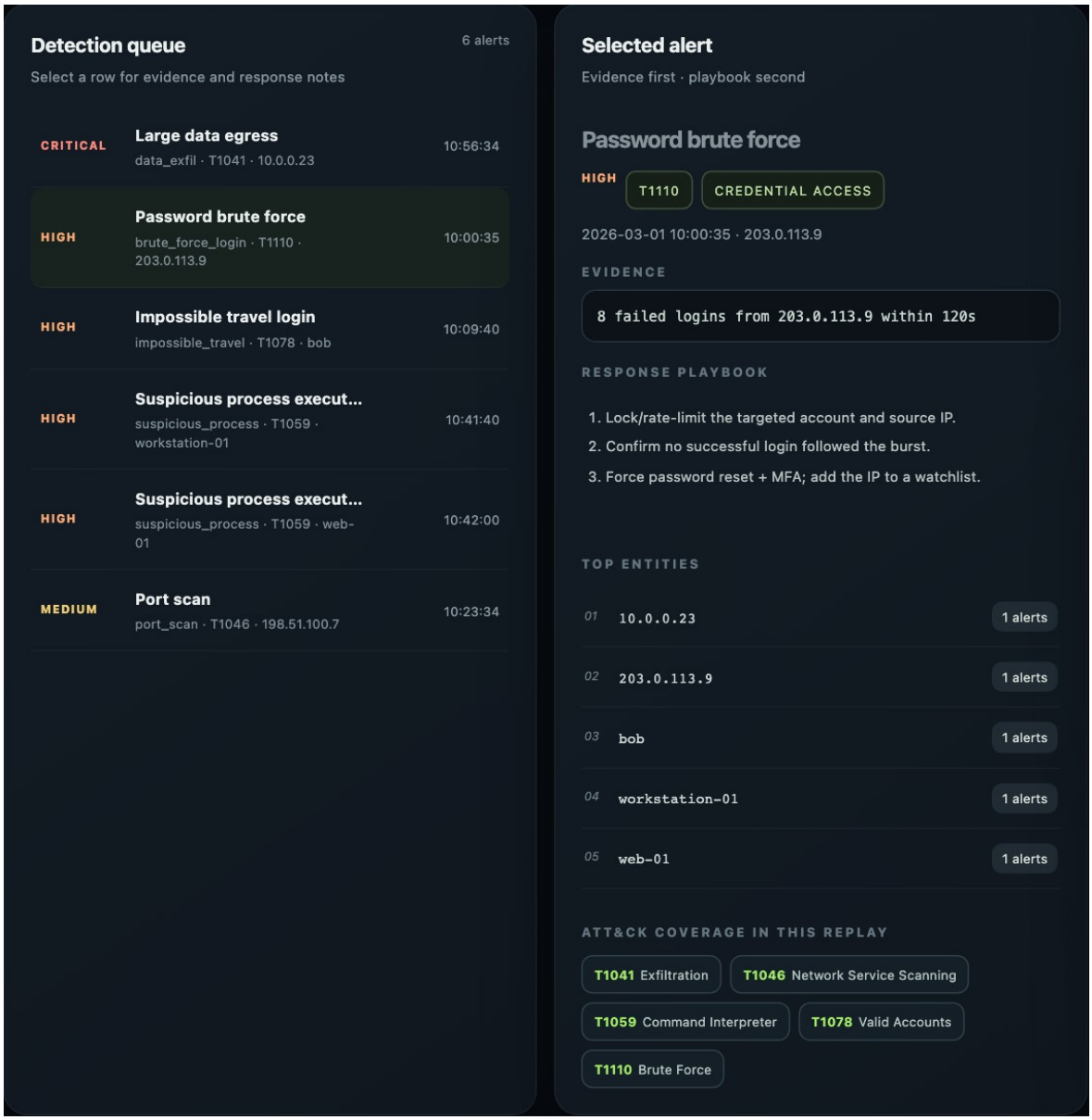


Figure 7.2: Alert triage with evidence and response playbook

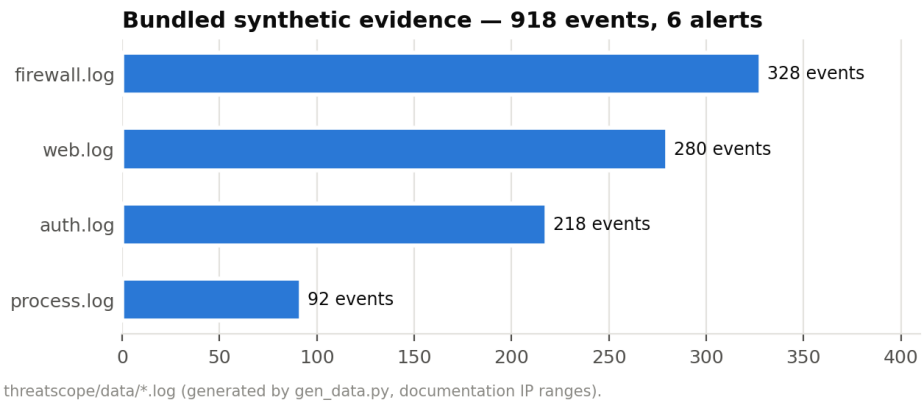


Figure 7.3: Bundled synthetic evidence by source

7.2 Real Tool Integration

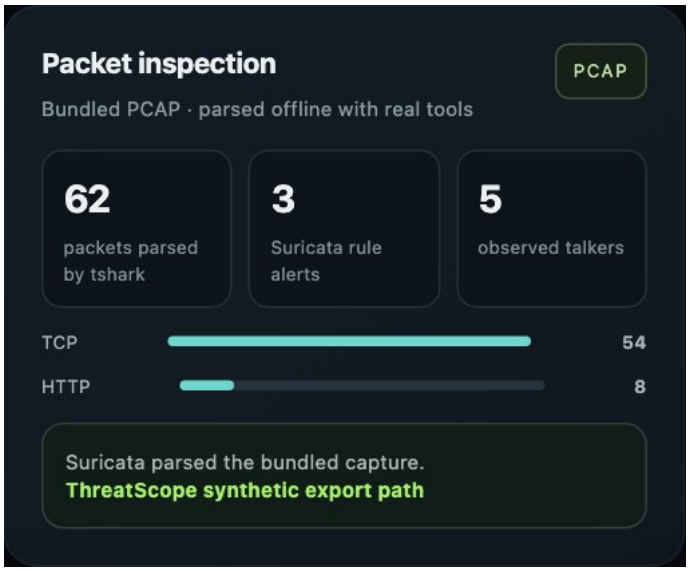


Figure 7.4: TShark (62 packets) and Suricata (3 alerts) on the bundled PCAP

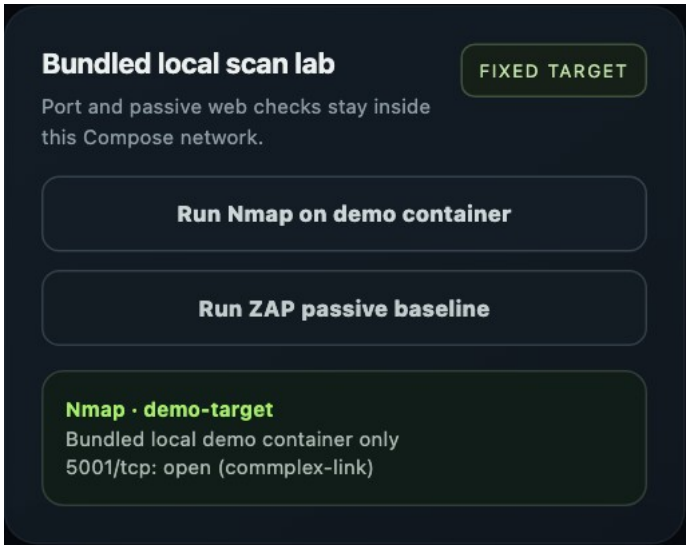


Figure 7.5: Nmap connect scan of the isolated demo target

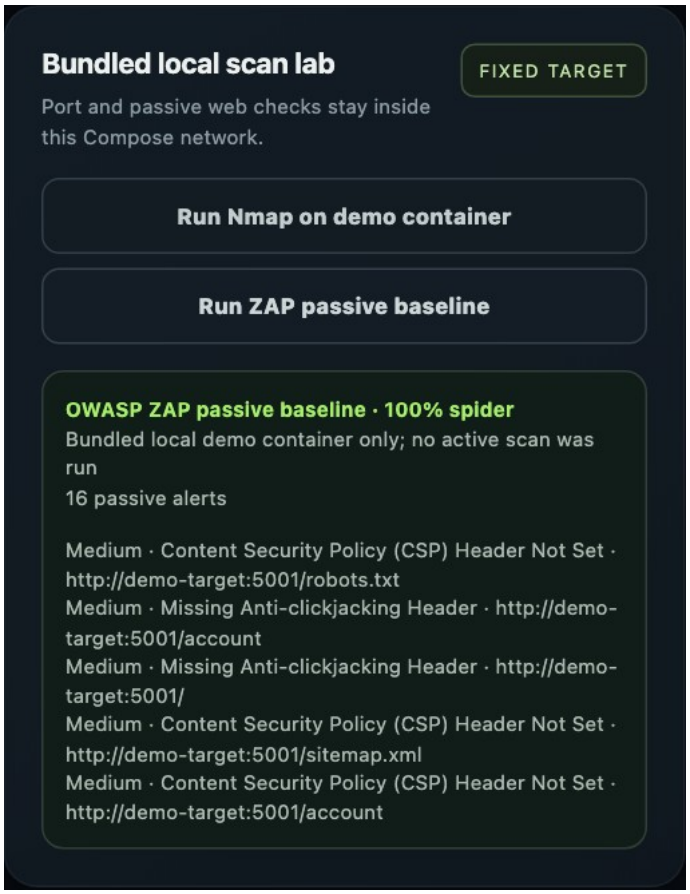


Figure 7.6: OWASP ZAP spider and passive baseline

Table 7.1: ZAP passive findings on the demo target (16 alerts)

Finding	Risk	Count
Content Security Policy (CSP) Header Not Set	Medium	5
Missing Anti-clickjacking Header	Medium	3
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	5
X-Content-Type-Options Header Missing	Low	3

The demo target is deliberately unhardened. Every ZAP finding (missing CSP, missing anti-clickjacking header, version disclosure, missing nosniff) corresponds to a control that ThreatScope's own `web_security` layer applies, so the lab demonstrates both finding and fixing the issue.

7.3 Ground-Truth Detection Quality

Table 7.2: Per-rule results over 25 seeds (default thresholds)

Rule	TP	FP	FN	Precision	Recall
brute_force_login	25	0	0	1.00	1.00
data_exfil	25	0	0	1.00	1.00
impossible_travel	25	0	0	1.00	1.00

Rule	TP	FP	FN	Precision	Recall
port_scan	25	0	0	1.00	1.00
suspicious_process	50	0	0	1.00	1.00
All rules	150	0	0	**1.00**	**1.00**

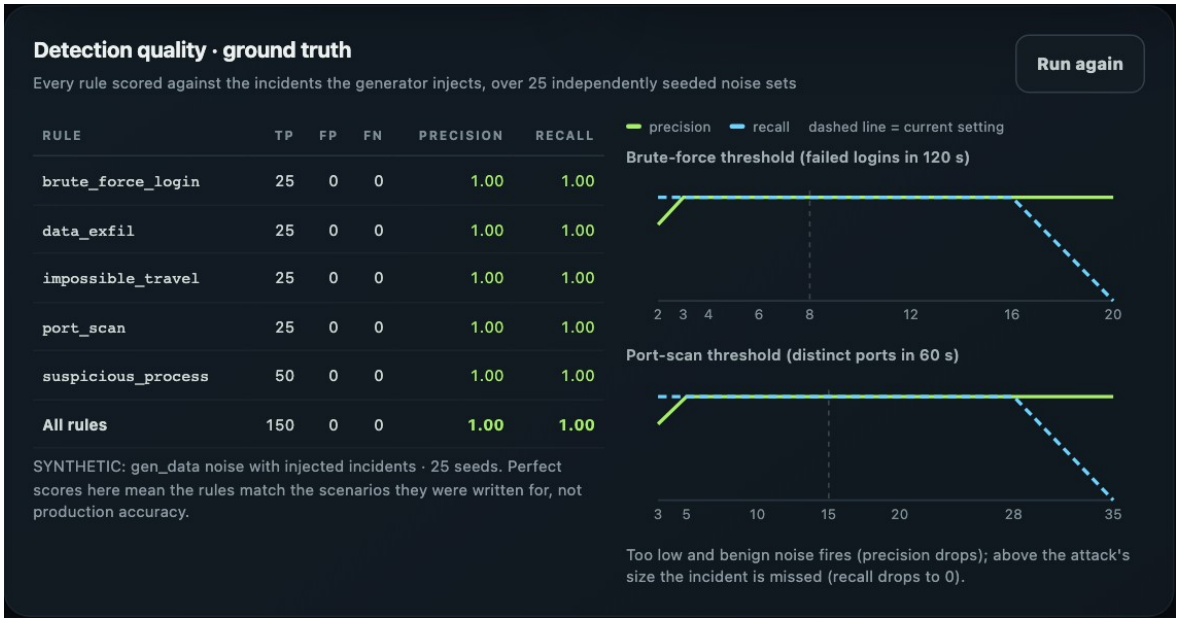


Figure 7.7: Detection-quality benchmark in the console

At default thresholds every injected incident was detected in every seed (150 TP) with no false positives. This shows the rules behave as designed on the scenarios they target; it is not evidence of real-world accuracy, because the benign noise is simple and the rules were written with these scenarios in mind [7].

7.4 Threshold Sensitivity

Table 7.3: Brute-force threshold sweep (failures per 120 s)

Threshold	FP	FN	Precision	Recall
2	9	0	0.73	1.00
3	0	0	1.00	1.00
4	0	0	1.00	1.00
6	0	0	1.00	1.00
8	0	0	1.00	1.00
12	0	0	1.00	1.00
16	0	0	1.00	1.00
20	0	25	1.00	0.00

Table 7.4: Port-scan threshold sweep (distinct ports per 60 s)

Threshold	FP	FN	Precision	Recall
3	9	0	0.73	1.00

Threshold	FP	FN	Precision	Recall
5	0	0	1.00	1.00
10	0	0	1.00	1.00
15	0	0	1.00	1.00
20	0	0	1.00	1.00
28	0	0	1.00	1.00
35	0	25	1.00	0.00

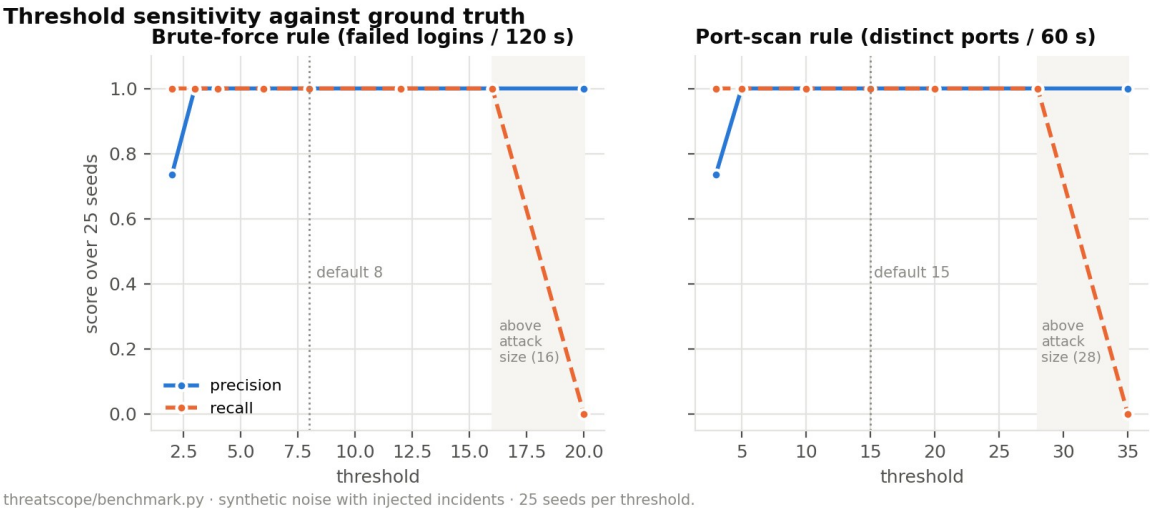


Figure 7.8: Precision and recall versus threshold

The sweeps reveal each rule's operating band. For brute force, a threshold of 2 produces 9 false positives over 25 seeds (benign users who mistype twice within two minutes), thresholds 3–16 are exact, and 20 exceeds the 16-failure attack so recall drops to 0. For port scans, 3 adds 9 false positives, 5–28 are exact, and 35 exceeds the 28-port sweep. The defaults (8 and 15) sit near the middle of these bands, which gives margin against noisier environments and slower attacks. Equally, the analysis shows that an attacker who stays below the threshold (e.g. 7 failures per window) evades the rule, an argument for layered detections.

7.5 Performance

Table 7.5: Load-test results (60 requests, 4 workers)

Endpoint	Req/s	p50 ms	p95 ms	max ms	Errors
full dashboard (918 events + tshark + Suricata)	30	126.9	143.5	163.8	0
health check	2,487	1.4	2.3	2.9	0

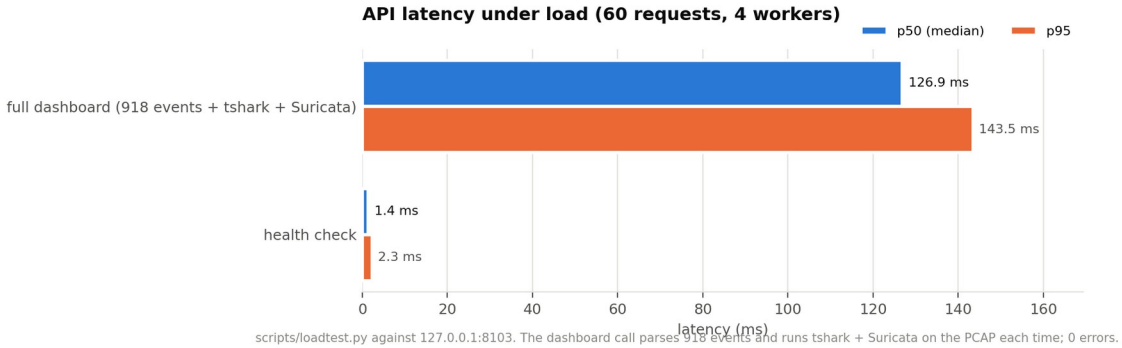


Figure 7.9: Latency per endpoint

The full dashboard call, which parses all 918 events, runs every rule and executes TShark and Suricata on the PCAP, has a median under 130 ms. Running the external tools dominates the cost; caching their output for the unchanged bundled PCAP would be an easy optimisation.

7.6 Mobile Layout

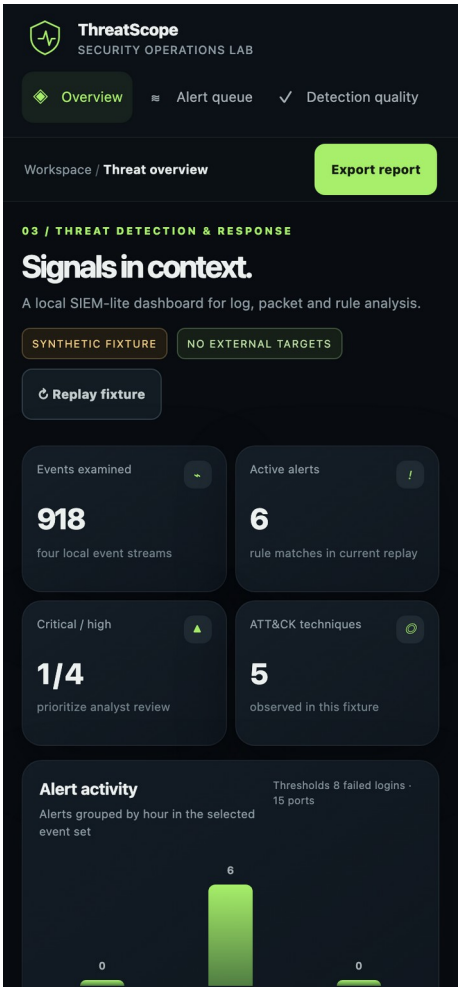


Figure 7.10: Console at 390 px width

7.7 Limitations

- Telemetry is synthetic and simple; perfect benchmark scores reflect that.

- Rules are hand-written, single-event-type detections; no cross-source correlation of one actor across logs yet.
- Impossible travel uses internal/external address change, not geolocation and velocity.
- Authentication is a single shared token; no user roles or per-analyst audit identity.
- Parsing a hostile PCAP exposes TShark/Suricata dissectors; mitigated by container isolation and limits, not eliminated.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

ThreatScope delivers a working, safe SOC lab: five ATT&CK-mapped rules over four log sources, real packet and web tooling isolated by design, an analyst console, and, as its research contribution, a ground-truth benchmark with threshold sweeps that makes each rule's behaviour measurable. On 25 seeds the rules detected all 150 injected incidents without false positives, and the sweeps located each rule's safe operating band. Testing also uncovered and fixed a networking defect that made the console unreachable. All 13 automated tests pass.

As the third design project it combines the foundations of the first two (secure tool integration in CryptoGuard; evaluation design and threat modelling in PhishLens) into a platform-scale system.

8.2 Future Scope

- Express rules in Sigma and test them with the same benchmark harness.
- Add cross-source correlation (scan → brute force → exfil by one actor) and alert scoring.
- Replay a licensed public dataset with labelled attacks for external validity.
- Per-analyst accounts with roles, and case management for alerts.
- Cache tool output and stream large uploads; add Zeek for protocol logs.

REFERENCES

[1] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington and C. B. Thomas, “MITRE ATT&CK: Design and Philosophy,” The MITRE Corporation, Technical Report, 2018.

[2] The MITRE Corporation, “MITRE ATT&CK knowledge base.” [Online]. Available: https://attack.mitre.org/

[3] P. Cichonski, T. Millar, T. Grance and K. Scarfone, “Computer Security Incident Handling Guide,” NIST Special Publication 800-61 Rev. 2, 2012.

[4] K. Kent and M. Souppaya, “Guide to Computer Security Log Management,” NIST Special Publication 800-92, 2006.

[5] D. E. Denning, “An intrusion-detection model,” IEEE Transactions on Software Engineering, vol. SE-13, no. 2, pp. 222–232, 1987.

[6] M. Roesch, “Snort: lightweight intrusion detection for networks,” in Proc. 13th USENIX Systems Administration Conference (LISA), 1999, pp. 229–238.

[7] R. Sommer and V. Paxson, “Outside the closed world: on using machine learning for network intrusion detection,” in Proc. IEEE Symposium on Security and Privacy, 2010, pp. 305–316.

[8] V. Paxson, “Bro: a system for detecting network intruders in real-time,” Computer Networks, vol. 31, no. 23–24, pp. 2435–2463, 1999.

[9] Open Information Security Foundation (OISF), “Suricata User Guide.” [Online]. Available: https://docs.suricata.io/

[10] Wireshark Foundation, “tshark — Dump and analyze network traffic,” manual page. [Online]. Available: https://www.wireshark.org/docs/man-pages/tshark.html

[11] G. F. Lyon, Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning. Sunnyvale, CA, USA: Insecure.Com LLC, 2009.

[12] ZAP Project, “ZAP documentation.” [Online]. Available: https://www.zaproxy.org/docs/

[13] SigmaHQ, “Sigma — generic signature format for SIEM systems.” [Online]. Available: https://github.com/SigmaHQ/sigma

[14] Wazuh, Inc., “Wazuh documentation.” [Online]. Available: https://documentation.wazuh.com/

[15] J. Arkko, M. Cotton and L. Vegoda, “IPv4 Address Blocks Reserved for Documentation,” IETF RFC 5737, 2010.

[16] OWASP Foundation, “OWASP Secure Headers Project.” [Online]. Available: https://owasp.org/www-project-secure-headers/

[17] A. Khraisat, I. Gondal, P. Vamplew and J. Kamruzzaman, “Survey of intrusion detection systems: techniques, datasets and challenges,” Cybersecurity, vol. 2, art. 20, 2019.

APPENDIX A: INSTALLATION AND USER MANUAL

8. Install Docker Desktop (allow ≈ 8 GB RAM).
9. Run `docker compose up --build -d` in the project folder (first run downloads ZAP, ≈ 1.5 GB).
10. Open http://localhost:8103.
11. Optional: set THREATSCOPE_API_TOKEN in .env to require a token; the console will prompt for it.
12. Stop with `docker compose down`.

Listing A.1: Developer commands

```
python3 -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt
make test         # 13 tests
make evaluate     # fixture rule counts
make benchmark    # ground truth + sweeps
make run          # http://127.0.0.1:8000 (tools show as unavailable outside Docker)
python scripts/loadtest.py
```

Table A.2: Troubleshooting

Symptom	Fix
Console unreachable on 8103	Pull the latest compose file (web network fix); docker compose up -d
ZAP card unavailable	Wait for the ZAP container to become healthy, or set THREATSCOPE_ZAP_ENABLED=0
Token prompt appears	THREATSCOPE_API_TOKEN is set; enter it, or unset it for loopback use
429 on scans	Rate limits (Nmap 4/min, ZAP 2/min) working as designed

APPENDIX B: DEMONSTRATION SCRIPT

13. Overview: KPIs count up; 918 events, 6 alerts, 5 techniques. State that the data are synthetic.
14. Open the critical exfiltration alert: evidence, ATT&CK tag, playbook steps.
15. Move the failed-login slider to 20 and replay: the brute-force alert disappears; explain thresholds.
16. Packet inspection: TShark and Suricata results on the bundled PCAP.
17. Run Nmap, then ZAP; explain the fixed target and the internal network.
18. Run the detection-quality benchmark: 150 TP, 0 FP; read the threshold bands from the chart.
19. Close on safety and on measured detection engineering.

APPENDIX C: API REFERENCE

Table C.1: REST endpoints

Method and path	Input	Output
GET /api/dashboard	brute_force, port_scan (query)	alerts, summary, timeline, techniques, pcap, suricata
POST /api/upload	multipart files[], thresholds	same shape as dashboard, label USER-UPLOADED
GET /api/benchmark	—	per_rule, overall, sweeps
POST /api/nmap-demo	— (target fixed)	ports, open_count
POST /api/zap-demo	— (target fixed)	alerts, alert_count, spider_percent
GET /health	—	ok (never requires a token)

APPENDIX D: REPOSITORY STRUCTURE

Listing D.1: Repository layout

```
├── .dockerignore
├── .env.example
├── .github/
│   └── workflows/
│       └── ci.yml
├── .gitignore
├── CHANGELOG.md
├── CONTRIBUTING.md
├── Dockerfile
├── LICENSE
├── Makefile
├── PROJECT_OVERVIEW.md
├── README.md
├── SECURITY.md
├── Start.command
├── docker-compose.yml
├── docs/
│   ├── README.md
│   ├── diagrams/  (11 images)
│   ├── index.html
│   └── screenshots/  (7 images)
├── requirements.txt
├── scripts/
│   └── loadtest.py
├── tests/
│   ├── test_security.py
│   └── test_threatscope.py
└── threatscope/
    ├── __init__.py
    ├── app.py
    ├── benchmark.py
    ├── data/
    │   ├── auth.log
    │   ├── firewall.log
    │   ├── process.log
    │   ├── sample-traffic.pcap
    │   └── web.log
    ├── demo_target.py
    ├── detect.py
    ├── evaluate.py
    ├── gen_data.py
    ├── nettools.py
    ├── rules/
    │   └── local-demo.rules
    ├── static/
    │   └── index.html
    └── web_security.py
```